

Proyecto Servlet

Andres Felipe Galindo Quiñones

Andres David Franco Galindo

Universidad Manuela Beltrán

Taller De Programación

Deivy Arley Torres Carreño

10 de mayo de 2025

Introducción	3
Crear el Servlet	4
ServletTienda	4
web.xml.....	6
Index.html	8
Servlet de compra	11
Catalogo HTML.....	13
Servlet de agregar producto	16
Implementación de CRUD.....	17
Cuenta HTML	20
Conexión Con base de datos	22
Base de datos en MySQL.....	22
Ejecución.....	25
Conclusión	29
Referencias.....	30

Introducción

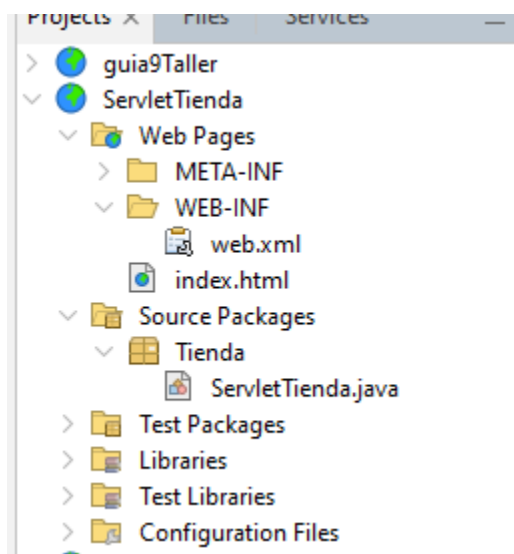
El proyecto actual busca desarrollar una aplicación web con servlets, centrada en una tienda de ropa. Su principal funcionalidad es permitir que el usuario se registre a través de una página de inicio diseñada en HTML. El proyecto se centra en garantizar que el proceso de registro del usuario sea funcional, almacenando sus datos en una base de datos. Garantizar la persistencia de estos datos es fundamental para su posterior consulta.

A continuación, se redirige al usuario a una página donde puede ver todos los productos disponibles en la tienda. Al seleccionar un artículo, aparece una ventana que le informa que ha sido comprado. Esta es una experiencia intuitiva e interactiva para el usuario. Una función más reciente permite al usuario eliminar su cuenta, si lo desea, mediante las funciones estándar de la aplicación asociadas con la base de datos: CRUD (Crear, Leer, Actualizar y Eliminar).

Crear el Servlet

Iniciando con el proyecto principalmente debemos crear un servlet en Java en cual iniciamos creando el proyecto con web application y le damos como nombre a la carpeta de ServletTienda como podemos ver en la Figura No1. Debido a que nuestra temática va ser enfocada en una tienda online de ropa, y creamos también la estructura de web.xml y el index.html, como se evidencia a continuación

Figura No.1. Estructura



Fuente: Autoría propia

ServletTienda

En la figura No.2 es un servlet Java (ServletTienda) recibe datos de un formulario HTML (nombre, email, contraseña y dirección), los guarda en una base de datos MySQL en la tabla usuarios1 y, si la inserción es exitosa, guarda el ID generado del usuario en la sesión HTTP. Luego redirige al usuario a catalogo.html. Si ocurre un error, se muestra un mensaje en el navegador.

Figura No.2. ServletTienda

```

1 package Tienda;
2 // Declara que este servlet pertenece al paquete "Tienda".
3
4 import java.io.*;
5 // Importa clases para manejar entradas y salidas (ej. PrintWriter).
6
7 import java.sql.*;
8 // Importa clases necesarias para trabajar con bases de datos (JDBC).
9
10 import jakarta.servlet.*;
11 // Importa clases base para Servlets.
12
13 import jakarta.servlet.http.*;
14 // Importa clases para trabajar con funcionalidades HTTP (como sesiones y peticiones).
15
16 public class ServletTienda extends HttpServlet {
17 // Define una clase servlet que extiende HttpServlet para manejar peticiones web.
18
19     protected void processRequest(HttpServletRequest request, HttpServletResponse response)
20         throws ServletException, IOException {
21         // Método que procesa tanto peticiones GET como POST.
22
23         String nombre = request.getParameter("nombre");
24         String email = request.getParameter("email");
25         String password = request.getParameter("passwd");
26         String direccion = request.getParameter("direccion");
27         // Obtiene los datos del formulario HTML enviados por el cliente.
28
29         response.setContentType("text/html;charset=UTF-8");
30         // Define el tipo de contenido de la respuesta como HTML con codificación UTF-8.
31
32         try (PrintWriter out = response.getWriter()) {

```

```

33             // Abre un flujo de escritura para enviar datos al cliente.
34
35             try (Connection conn = ConexionBD.obtenerConexion()) {
36                 // Abre una conexión a la base de datos usando una clase personalizada ConexionBD.
37
38                 String sql = "INSERT INTO usuarios1 (nombre, email, contrasena, direccion) VALUES (?, ?, ?, ?)";
39                 // Define la consulta SQL para insertar un nuevo usuario.
40
41                 PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
42                 // Prepara el statement y solicita que devuelva las claves generadas (ID autoincremental).
43
44                 stmt.setString(1, nombre);
45                 stmt.setString(2, email);
46                 stmt.setString(3, password);
47                 stmt.setString(4, direccion);
48                 // Asigna los valores obtenidos del formulario a los placeholders de la consulta SQL.
49
50                 int filas = stmt.executeUpdate();
51                 // Ejecuta la consulta de inserción. Devuelve el número de filas afectadas.
52
53                 if (filas > 0) {
54                     // Si se insertó al menos una fila (registro exitoso):
55
56                     ResultSet rs = stmt.getGeneratedKeys();
57                     if (rs.next()) {
58                         int usuarioId = rs.getInt(1);
59                         // Recupera el ID generado automáticamente por la base de datos.
60
61                         HttpSession session = request.getSession();
62                         session.setAttribute("usuario_id", usuarioId);
63                         // Guarda ese ID en la sesión HTTP para futuras referencias del usuario.
64                     }
65                 }
66             }
67         }
68     }
69 }

```

```

// Guarda ese ID en la sesión HTTP para futuras referencias del usuario.
}
response.sendRedirect("catalogo.html");
// Redirige al usuario al catálogo después del registro.
} else {
    out.println("<h2>Error al registrar usuario</h2>");
    // Si no se insertó ninguna fila, muestra un mensaje de error.
}

} catch (SQLException e) {
    out.println("<h2>Error de conexión: " + e.getMessage() + "</h2>");
    // Captura errores relacionados con la base de datos y los muestra.
}
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
    // Llama a processRequest cuando se recibe una petición POST.
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
    // Llama a processRequest cuando se recibe una petición GET.
}
}

```

web.xml

En la Figura No.3 es un archivo en donde es el descriptor de despliegue de una aplicación web en Jakarta EE (anteriormente Java EE). Define la configuración de los servlets, es decir, qué clases se ejecutan en respuesta a ciertas URLs, así como páginas por defecto y páginas de error. En este caso, registra tres servlets (ServletTienda, CompraServlet y UsuarioCRUDServlet), define qué URL activa a cada uno, y establece una página para errores 404, además de los archivos de bienvenida que se mostrarán al acceder sin ruta específica.

Figura No.3 Archivo web.xmlm

```

<?xml version="1.0" encoding="UTF-8"?>
<!-- Declaración del archivo XML. -->

<web-app version="5.0"
  xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app 5.0.xsd">

  <!-- Inicio del descriptor web. Define versión y esquema XML usado para Jakarta EE 10 (v5.0). -->

  <!-- Configuración de servlets -->
  <servlet>
    <servlet-name>ServletTienda</servlet-name>
    <servlet-class>Tienda.ServletTienda</servlet-class>
  </servlet>
  <!-- Registra el servlet llamado "ServletTienda", indicando su clase Java. -->

  <servlet-mapping>
    <servlet-name>ServletTienda</servlet-name>
    <url-pattern>/ServletTienda</url-pattern>
  </servlet-mapping>
  <!-- Mapea la URL "/ServletTienda" para que sea manejada por el servlet "ServletTienda". -->

  <servlet>
    <servlet-name>CompraServlet</servlet-name>
    <servlet-class>Tienda.ServletCompra</servlet-class>
  </servlet>
  <!-- Registra otro servlet, "CompraServlet", de la clase correspondiente. -->

  <servlet-mapping>
    <servlet-name>CompraServlet</servlet-name>
    <url-pattern>/CompraServlet</url-pattern>

    <url-pattern>/CompraServlet</url-pattern>
  </servlet-mapping>
  <!-- Mapea la URL "/CompraServlet" al servlet "CompraServlet". -->

  <servlet>
    <servlet-name>UsuarioCRUDServlet</servlet-name>
    <servlet-class>Tienda.UsuarioCRUDServlet</servlet-class>
  </servlet>
  <!-- Registra el servlet que maneja operaciones CRUD sobre usuarios. -->

  <servlet-mapping>
    <servlet-name>UsuarioCRUDServlet</servlet-name>
    <url-pattern>/UsuarioCRUD</url-pattern>
  </servlet-mapping>
  <!-- Mapea la URL "/UsuarioCRUD" a "UsuarioCRUDServlet". -->

  <!-- Configuración de páginas de error -->
  <error-page>
    <error-code>404</error-code>
    <location>/error404.jsp</location>
  </error-page>
  <!-- Cuando ocurre un error 404 (página no encontrada), redirige al archivo JSP personalizado. -->

  <!-- Configuración de páginas por defecto -->
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>index.html</welcome-file>
    <welcome-file>login.jsp</welcome-file>
  </welcome-file-list>
  <!-- Define archivos que se mostrarán por defecto cuando se accede al sitio raíz. -->

</web-app>

```

Fuente: Autoría Propia

Index.html

En la Figura No.4 podemos evidenciar un archivo HTML muestra un formulario visualmente atractivo para registrar usuarios en una tienda de ropa. Recoge datos como nombre, correo, contraseña y dirección, y los envía al servlet ServletTienda mediante el método POST. El diseño está centrado y estilizado con CSS. Incluye funcionalidad en JavaScript para mostrar/ocultar la contraseña y para mostrar un mensaje si la cuenta fue eliminada (mediante un parámetro en la URL)

Figura No.4. Archivo Index.Html

```
<!DOCTYPE html> <!-- Define el tipo de documento como HTML5 -->
<html lang="es"> <!-- Especifica el idioma del contenido como español -->

<head> <!-- Comienza la sección de encabezado del documento -->
  <meta charset="UTF-8"> <!-- Establece la codificación de caracteres como UTF-8 -->
  <title>Registro Tienda de Ropa</title> <!-- Título que se mostrará en la pestaña del navegador -->

  <style> <!-- Inicio del bloque de estilos CSS -->

    /* Estilos generales del cuerpo de la página */
    body {
      font-family: Arial, sans-serif; /* Fuente predeterminada */
      background: linear-gradient(to right, #74ebd5, #ACB6E5); /* Fondo con degradado de color */
      height: 100vh; /* Altura del cuerpo igual al alto de la ventana */
      display: flex; /* Usa flexbox para el contenido */
      justify-content: center; /* Centra horizontalmente */
      align-items: center; /* Centra verticalmente */
      margin: 0; /* Elimina márgenes predeterminados */
    }

    /* Estilo del contenedor del formulario */
    .form-container {
      background: #ffffff; /* Fondo blanco */
      padding: 30px 40px; /* Espaciado interno */
      border-radius: 15px; /* Bordes redondeados */
      box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2); /* Sombra alrededor del contenedor */
      width: 300px; /* Ancho fijo */
      text-align: center; /* Centrado del texto */
      position: relative; /* Posición relativa para usar elementos absolutos dentro */
    }
  </style>
</head>

<body>
  <div class="form-container">
    <h2>Registro de Usuario</h2>
    <input type="text" value="Nombre" />
    <input type="text" value="Correo" />
    <input type="password" value="Contraseña" />
    <input type="text" value="Dirección" />
    <input type="button" value="Registrar" />
  </div>
</body>
```



```

/* Estilo de la alerta de mensaje */
.alert {
    background-color: #d4edda; /* Fondo verde claro */
    color: #155724; /* Texto verde oscuro */
    padding: 10px; /* Espaciado interno */
    border-radius: 5px; /* Bordes redondeados */
    border: 1px solid #c3e6cb; /* Borde de alerta */
    margin-bottom: 15px; /* Espacio inferior */
    font-size: 14px; /* Tamaño del texto */
}

/* Estilo del título principal */
h2 {
    margin-bottom: 20px; /* Espaciado inferior */
    color: #333; /* Color de texto oscuro */
}

/* Estilo para las etiquetas de los campos */
label {
    display: block; /* Muestra cada etiqueta en su propia línea */
    text-align: left; /* Alinea el texto a la izquierda */
    margin-top: 15px; /* Margen superior */
    color: #444; /* Color del texto */
}

/* Estilos para los campos de entrada de texto, email y contraseña */
input[type="text"],
input[type="email"],
input[type="password"] {
    width: 100%; /* Ocupa el 100% del ancho disponible */
    padding: 10px; /* Espaciado interno */

```

```

    margin-top: 5px; /* Margen superior */
    border: 1px solid #ccc; /* Borde gris claro */
    border-radius: 8px; /* Bordes redondeados */
}

/* Contenedor del campo de contraseña para ubicar el botón de mostrar */
.password-container {
    position: relative; /* Para posicionar el botón dentro de este contenedor */
}

/* Estilo del botón para mostrar/ocultar contraseña */
.toggle-password {
    position: absolute; /* Posición absoluta dentro del contenedor */
    right: 10px; /* Distancia del borde derecho */
    top: 37px; /* Posición vertical */
    cursor: pointer; /* Cambia el cursor al pasar por encima */
    font-size: 14px; /* Tamaño de fuente */
    color: #007BFF; /* Color azul */
}

/* Estilo del botón de enviar */
input[type="submit"] {
    margin-top: 20px; /* Margen superior */
    width: 100%; /* Ocupa todo el ancho */
    padding: 10px; /* Espaciado interno */
    background-color: #007BFF; /* Fondo azul */
    border: none; /* Sin borde */
    color: white; /* Texto blanco */
    font-size: 16px; /* Tamaño del texto */
    border-radius: 8px; /* Bordes redondeados */
    cursor: pointer; /* Cursor tipo botón */
}

```

```

border-radius: 8px; /* Bordes redondeados */
cursor: pointer; /* Cursor tipo botón */

/* Cambia el color del botón al pasar el mouse por encima */
input[type="submit"]:hover {
    background-color: #0056b3; /* Azul más oscuro al hacer hover */
}

</style> <!-- Fin del bloque de estilos -->
</head>

```

```

<body> <!-- Inicio del cuerpo de la página -->

<!-- Contenedor principal del formulario -->
<div class="form-container" id="form-container">

    <!-- Título del formulario -->
    <h2>Registro de Usuario</h2>

    <!-- Inicio del formulario, que envía datos al servlet llamado 'ServletTienda' -->
    <form action="ServletTienda" method="post">
        <!-- Campo de nombre -->
        <label>Nombre:</label>
        <input type="text" name="nombre" required> <!-- Campo obligatorio -->

        <!-- Campo de correo electrónico -->
        <label>Correo:</label>
        <input type="email" name="email" required> <!-- Campo obligatorio -->

        <!-- Campo de contraseña con opción de mostrar -->
        <label>Contraseña:</label>

```

```

        <label>Contraseña:</label>
        <div class="password-container"> <!-- Contenedor para mostrar botón de mostrar contraseña -->
            <input type="password" name="passwd" id="passwd" required> <!-- Campo obligatorio -->
            <span class="toggle-password" onclick="togglePassword()">Mostrar</span> <!-- Texto clickeable para mostrar la contraseña -->
        </div>

        <!-- Campo de dirección -->
        <label>Dirección:</label>
        <input type="text" name="direccion" required> <!-- Campo obligatorio -->

        <!-- Botón para enviar el formulario -->
        <input type="submit" value="Entrar">
    </form> <!-- Fin del formulario -->
</div> <!-- Fin del contenedor del formulario -->

<!-- Inicio del bloque de JavaScript -->
<script>
    // Función que alterna entre mostrar y ocultar la contraseña
    function togglePassword() {
        var passwdInput = document.getElementById("passwd"); // Obtiene el input de la contraseña
        var toggle = document.querySelector(".toggle-password"); // Obtiene el botón para mostrar/ocultar

        // Si el tipo es "password", lo cambia a "text"
        if (passwdInput.type === "password") {
            passwdInput.type = "text";
            toggle.textContent = "Ocultar"; // Cambia el texto del botón
        } else {
            passwdInput.type = "password";
            toggle.textContent = "Mostrar"; // Vuelve a cambiar el texto
        }
    }
}

```

```

// Revisa si la URL contiene el parámetro ?mensaje=eliminada
const params = new URLSearchParams(window.location.search); // Obtiene parámetros de la URL
if (params.get("mensaje") === "eliminada") { // Si mensaje es "eliminada"
    const alert = document.createElement("div"); // Crea un div nuevo
    alert.className = "alert"; // Asigna la clase de alerta
    alert.textContent = "Cuenta eliminada exitosamente"; // Mensaje que se mostrará
    const container = document.getElementById("form-container"); // Obtiene el contenedor
    container.insertBefore(alert, container.firstChild); // Inserta el mensaje antes del primer hijo
}

</script> <!-- Fin del bloque JavaScript -->
</body> <!-- Fin del cuerpo -->
</html> <!-- Fin del documento HTML -->

```

Fuente: Autoría Propia

Servlet de compra

Una vez el usuario realiza se registra del producto en la Figura No 5 podemos ver un servlet procesa una compra de un producto por parte de un usuario en una tienda online. Al recibir un POST, busca al usuario por su correo y obtiene su ID y dirección. Luego busca el producto por su nombre y también obtiene su ID. Si ambos existen, inserta una nueva compra en la base de datos y muestra una página HTML de confirmación de compra con un diseño estilizado. Si hay errores (usuario o producto no encontrados, o fallos de conexión), muestra mensajes adecuados.

Figura No.5. Servlet de compra

```

package Tienda;
// Define el paquete donde se encuentra el servlet

import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import static java.lang.System.out;
// Importa clases necesarias para manejar E/S, base de datos, y HTTP

public class ServletCompra extends HttpServlet {
    // Define el servlet para manejar compras

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Método que se ejecuta al recibir un formulario por POST

        String email = request.getParameter("email");
        String producto = request.getParameter("producto"); // Puede ser "Camisa", "Jean", "Zapatos"
        // Recupera los datos enviados desde el formulario

        response.setContentType("text/html;charset=UTF-8");
        // Define el tipo de contenido de la respuesta

        try (PrintWriter out = response.getWriter();
            Connection conn = ConexionBD.obtenerConexion()) {
            // Crea un escritor de salida y establece la conexión con la base de datos

            // Busca al usuario por correo
            PreparedStatement psUser = conn.prepareStatement("SELECT id, direccion FROM usuarios1 WHERE email = ?");
            psUser.setString(1, email);

```

```

psUser.setString(1, email);
ResultSet rsUser = psUser.executeQuery();

if (!rsUser.next()) {
    // Si no encuentra el usuario, muestra mensaje y termina
    out.println("<div style='text-align:center; background-color:#ffebee; padding:20px; border-radius:10px;'>"
        + "<h2 style='color:#d32f2f;'>Usuario no encontrado</h2></div>");
    return;
}

int usuarioId = rsUser.getInt("id");
String direccion = rsUser.getString("direccion");
// Recupera el ID del usuario y su dirección

// Busca el producto por nombre
PreparedStatement psProd = conn.prepareStatement("SELECT id FROM productos WHERE nombre = ?");
psProd.setString(1, producto);
ResultSet rsProd = psProd.executeQuery();

if (!rsProd.next()) {
    // Si no encuentra el producto, muestra mensaje
    out.println("<div style='text-align:center; background-color:#ffebee; padding:20px; border-radius:10px;'>"
        + "<h2 style='color:#d32f2f;'>Producto no encontrado</h2></div>");
    return;
}

int productoId = rsProd.getInt("id");
// Obtiene el ID del producto

// Inserta la compra en la tabla "compras"
PreparedStatement psCompra = conn.prepareStatement(
    "INSERT INTO compras (usuario_id, producto_id) VALUES (?, ?)");

PreparedStatement psCompra = conn.prepareStatement(
    "INSERT INTO compras (usuario_id, producto_id) VALUES (?, ?)");
psCompra.setInt(1, usuarioId);
psCompra.setInt(2, productoId);
psCompra.executeUpdate();

// Muestra una página de confirmación visualmente atractiva
out.println("<!DOCTYPE html>");
out.println("<html>");
out.println("<head>");
out.println("<title>Compra Exitosa</title>");
out.println("<style>");
out.println("body { font-family: Arial, sans-serif; background-color: #f5f5f5; margin: 0; padding: 20px; }");
out.println(".container { max-width: 600px; margin: 50px auto; background: white; padding: 30px; border-radius:");
out.println(".success-icon { color: #4CAF50; font-size: 48px; margin-bottom: 20px; }");
out.println("h2 { color: #4CAF50; font-size: 28px; margin-bottom: 20px; }");
out.println(".product-info { background-color: #f9f9f9; padding: 15px; border-radius: 8px; margin: 20px 0; }");
out.println(".address { font-weight: bold; color: #333; }");
out.println(".reminder { margin-top: 25px; padding-top: 15px; border-top: 1px solid #eee; color: #666; }");
out.println("</style>");
out.println("</head>");
out.println("<body>");
out.println("<div class='container'>");
out.println("<div class='success-icon'></div>");
out.println("<h2>Compra exitosa!</h2>");
out.println("<div class='product-info'>");
out.println("<p>Has comprado una <strong> + producto + </strong>.</p>");
out.println("<p>Pronto llegará a tu casa: <span class='address'> + direccion + </span></p>");
out.println("</div>");
out.println("<div class='reminder'>");
out.println("<p>Recuerda pagar al domiciliario.</p>");

```

```

        out.println(".reminder { margin-top: 25px; padding-top: 15px; border-top: 1px solid #eee; color: #666; }");
        out.println("</style>");
        out.println("</head>");
        out.println("<body>");
        out.println("<div class='container'>");
        out.println("<div class='success-icon'>✓</div>");
        out.println("<h2>Compra exitosa!</h2>");
        out.println("<div class='product-info'>");
        out.println("<p>Has comprado una <strong>" + producto + "</strong></p>");
        out.println("<p>Pronto llegará a tu casa: <span class='address'>" + direccion + "</span></p>");
        out.println("</div>");
        out.println("<div class='reminder'>");
        out.println("<p>Recuerda pagar al domiciliario.</p>");
        out.println("</div>");
        out.println("</body>");
        out.println("</html>");
    } catch (SQLException e) {
        // Muestra error si hay algún problema en la base de datos
        out.println("<div style='text-align:center; background-color:#ffebee; padding:20px; border-radius:10px;'>"
            + "<h2 style='color:#d32f2f;'>Error en la compra: " + e.getMessage() + "</h2></div>");
    }
}

```

Fuente: Autoría Propia

Catalogo HTML

En la Figura No.6 es una página web muestra un catálogo visual con tres productos: camisa, jean y zapatos. Cada producto tiene su imagen, precio y un formulario para comprarlo, solicitando el email del usuario. Al hacer clic en "Agregar", el formulario envía los datos al servlet CompraServlet. También hay un botón en el encabezado para redirigir a la página de modificación o eliminación de cuenta

Figura No.6 Catalogo HTML

```

<!DOCTYPE html>
<!-- Indica que el documento está escrito en HTML5 -->

<html lang="es">
<!-- Inicio del documento HTML y se especifica que el idioma es español -->

<head>
    <meta charset="UTF-8">
    <!-- Establece la codificación de caracteres como UTF-8, para admitir caracteres especiales -->

    <title>Catálogo de Productos</title>
    <!-- Título que aparecerá en la pestaña del navegador -->

    <style>
        /* Comienzo de los estilos CSS */

        body {
            font-family: 'Segoe UI', sans-serif;
            background-color: #f0f8ff;
            margin: 0;
            padding: 0;
        }

        /* Estilo del cuerpo: fuente moderna, fondo azul claro, sin márgenes o relleno */

        header {
            background-color: #ffffff;
            padding: 15px 20px;
            display: flex;
            justify-content: space-between;
            align-items: center;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }
    </style>

```

```

/* Encabezado blanco con espacio interno, distribución horizontal y sombra inferior */

h1 {
    margin: 0;
    color: #222;
}

/* Título principal sin márgenes y con texto en gris oscuro */

.modificar-btn {
    padding: 10px 20px;
    background-color: orange;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
}

/* Botón de modificación de cuenta: naranja, texto blanco, redondeado y clickeable */

.catalogo {
    text-align: center;
    padding: 20px;
}

/* Contenedor principal del catálogo: centrado y con relleno */

.producto {
    border: 2px solid #007bff;
    border-radius: 10px;
    padding: 20px;
    margin: 15px;
    background-color: #ffffff;
    display: inline-block;

```

```

/* Tarjeta de producto: borde azul, fondo blanco, borde redondeado, sombra y en línea */

img {
    width: 150px;
    height: auto;
    border-radius: 5px;
}

/* Imágenes de los productos: ancho fijo, altura automática y bordes suavizados */

input[type="submit"] {
    margin-top: 10px;
    background-color: #28a745;
    color: white;
    border: none;
    padding: 8px 16px;
    border-radius: 5px;
    cursor: pointer;
}

/* Botón de enviar: verde, texto blanco, redondeado y clickeable */

input[type="email"] {
    padding: 5px;
    width: 90%;
    margin-top: 8px;
}

/* Campo de entrada de correo: relleno interno, ancho casi completo y separación superior */

</style>
</head>

<body>
<!-- Inicio del cuerpo del documento -->

```

```

<!-- Inicio del cuerpo del documento -->

<header>
  <!-- Encabezado superior de la página -->

  <h1>Catálogo de Productos</h1>
  <!-- Título del sitio -->

  <form method="get" action="UsuarioCRUD">
    <!-- Formulario que redirige al servlet de modificación/eliminación de cuenta -->
    <button type="submit" class="modificar-btn">Modificar o Eliminar Cuenta</button>
    <!-- Botón estilizado para acceder a la gestión de cuenta -->
  </form>
</header>

<div class="catalogo">
  <!-- Contenedor general del catálogo de productos -->

  <div class="producto">
    <!-- Primera tarjeta de producto -->

    <h2>Camisa</h2>
    <!-- Nombre del producto -->

    
    <!-- Imagen del producto con descripción alternativa -->

    <p>Precio: $50.000</p>
    <!-- Precio del producto -->

    <form action="CompraServlet" method="post">
      <!-- Botón para enviar el formulario y realizar la compra -->
    </form>
  </div>

  <div class="producto">
    <!-- Segunda tarjeta de producto -->

    <h2>Jean</h2>
    
    <p>Precio: $70.000</p>
    <form action="CompraServlet" method="post">
      <input type="hidden" name="producto" value="Jean">
      <input type="email" name="email" placeholder="Tu email" required><br>
      <input type="submit" value="Agregar">
    </form>
  </div>

  <div class="producto">
    <!-- Tercera tarjeta de producto -->

    <h2>Zapatos</h2>
    
    <p>Precio: $100.000</p>
    <form action="CompraServlet" method="post">
      <input type="hidden" name="producto" value="Zapatos">
      <input type="email" name="email" placeholder="Tu email" required><br>
      <input type="submit" value="Agregar">
    </form>
  </div>
</div>
</body>
</html>

```

Fuente: Autoría propia

Servlet de agregar producto

En la Figura No.7 es un servlet de AgregarProducto recibe una solicitud POST desde un formulario con el nombre de un producto, se conecta a la base de datos utilizando la clase ConexionBD y ejecuta una instrucción SQL para insertar ese nombre en la tabla productos. Si la operación es exitosa, muestra un mensaje en HTML confirmando que el producto fue agregado, junto con un enlace para volver al catálogo; en caso contrario, muestra un mensaje de error indicando que no se pudo agregar el producto o que hubo un problema de conexión.

Figura No.7 Servlet Agrega Producto

```
package Tienda; // Define el paquete donde se encuentra esta clase

// Importación de clases necesarias
import java.io.*; // Para trabajar con entrada/salida, como PrintWriter
import java.sql.*; // Para manejar bases de datos (JDBC)
import jakarta.servlet.*; // Para funcionalidades de Servlets
import jakarta.servlet.http.*; // Para manejar peticiones/respuestas HTTP

// Clase AgregarProducto que extiende HttpServlet
public class AgregarProducto extends HttpServlet {

    // Método que maneja solicitudes POST (por ejemplo, desde un formulario)
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Obtiene el nombre del producto enviado desde el formulario
        String producto = request.getParameter("producto");

        // Define el tipo de contenido de la respuesta como HTML con codificación UTF-8
        response.setContentType("text/html;charset=UTF-8");

        // Se usa try-with-resources para manejar automáticamente el cierre de PrintWriter
        try (PrintWriter out = response.getWriter()) {

            // Segundo try para manejar la conexión a la base de datos
            try (Connection conn = ConexionBD.obtenerConexion()) {
                // Sentencia SQL para insertar un nuevo producto en la tabla 'productos'
                String sql = "INSERT INTO productos (nombre) VALUES (?)";
                PreparedStatement stmt = conn.prepareStatement(sql); // Prepara la consulta
                stmt.setString(1, producto); // Asigna el valor del producto al parámetro de la consulta

                int filas = stmt.executeUpdate(); // Ejecuta la consulta (INSERT) y devuelve cuántas filas se modificaron
            }
        }
    }
}
```



```

        int filas = stmt.executeUpdate(); // Ejecuta la consulta (INSERT) y devuelve cuántas filas se modificaron
    }

    if (filas > 0) {
        // Si se insertó al menos una fila, se muestra un mensaje de éxito
        out.println("<h2>Producto agregado: " + producto + "</h2>");
        out.println("<a href='catalogo.html'>Volver al catálogo</a>");
    } else {
        // Si no se insertó ninguna fila, se muestra un mensaje de error
        out.println("<h2>Error al agregar producto</h2>");
    }
} catch (SQLException e) {
    // En caso de error en la conexión o en la consulta, se muestra el mensaje
    out.println("<h2>Error de conexión: " + e.getMessage() + "</h2>");
}
}
}
}

```

Fuente: Autoría propia

Implementación de CRUD

En la figura No.8 es un servlet que permite a los usuarios modificar o eliminar su cuenta desde un formulario web. En el método doGet, genera un formulario HTML con estilos para que el usuario pueda editar sus datos o eliminar su cuenta. En el método doPost, valida la contraseña actual, actualiza los datos si se solicita o elimina la cuenta y redirige según el resultado.

Figura No.8 Implementación de CRUD

```

/* - En el método doGet: muestra un formulario HTML para modificar o eliminar la cuenta.
 * - En el método doPost: verifica la contraseña actual, actualiza los datos si se solicita, o elimina la cuenta.
 */

package Tienda;

import java.io.*; // Manejo de entrada/salida
import java.sql.*; // Conexión y manipulación de base de datos
import jakarta.servlet.*; // Funcionalidad general de servlets
import jakarta.servlet.http.*; // Funcionalidad específica de HTTP como sesiones y peticiones

public class UsuarioCRUDServlet extends HttpServlet {

    // Muestra el formulario HTML al usuario
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Define el tipo de contenido de la respuesta
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter(); // Permite escribir en la respuesta

        // Empieza el HTML
        out.println("<!DOCTYPE html>");
        out.println("<html><head><title>Cuenta</title>");
        out.println("<style>");
        out.println("body { font-family: Arial; background-color: #f5f5f5; display: flex; justify-content: center; align-items: center; height: 100%; }");
        out.println(".form-container { background: white; padding: 30px; border-radius: 10px; box-shadow: 0 0 10px rgba(0,0,0,0.1); width: 400px; }");
        out.println("input[type=text], input[type=email], input[type=password] { width: 100%; padding: 10px; margin: 8px 0; border: 1px solid #ccc; }");
        out.println("button { padding: 10px 20px; border: none; border-radius: 5px; background-color: #007bff; color: white; font-weight: bold; }");
        out.println("button[name='action'][value='eliminar'] { background-color: #dc3545; float: right; }");
    }
}

```

```

out.println("<button name='accion' {value='Eliminar' ; background-color: #d4edda; border-radius: 5px; }");
out.println("<.message { padding: 10px; margin: 15px 0; border-radius: 5px; }");
out.println("<.success { background-color: #d4edda; color: #155724; }");
out.println("<.error { background-color: #f8d7da; color: #721c24; }");
out.println("</style></head><body>");
out.println("<div class='form-container'>");
out.println("<h2>Modificar o Eliminar Cuenta</h2>");

```

```

// Mostrar mensaje de éxito o error, si existe
String mensaje = request.getParameter("mensaje");
if (mensaje != null) {
    String clase = "";
    String texto = "";

    switch(mensaje) {
        case "cuenta_actualizada":
            clase = "success";
            texto = "Cuenta actualizada exitosamente";
            break;
        case "cuenta_eliminada":
            clase = "success";
            texto = "Cuenta eliminada exitosamente";
            break;
        case "contrasena_incorrecta":
            clase = "error";
            texto = "La contraseña actual no es correcta";
            break;
        case "usuario_no_encontrado":
            clase = "error";
            texto = "Usuario no encontrado";
            break;
        case "error_actualizacion":

```

```

        case "error_actualizacion":
            clase = "error";
            texto = "Error al actualizar la cuenta";
            break;
        case "error_eliminacion":
            clase = "error";
            texto = "Error al eliminar la cuenta";
            break;
        case "error_servidor":
            clase = "error";
            texto = "Error en el servidor. Por favor, intente más tarde";
            break;
        default:
            clase = "error";
            texto = "Error al procesar la solicitud";
    }
}

```

```

out.println("<div class='message " + clase + "'> " + texto + "</div>");
}

```

```

// Formulario para editar o eliminar cuenta
out.println("<form method='post' action='UsuarioCRUD'>");
out.println("<label for='email'>Email:</label>");
out.println("<input type='email' id='email' name='email' required>");
out.println("<label for='nombre'>Nombre:</label>");
out.println("<input type='text' id='nombre' name='nombre'>");
out.println("<label for='direccion'>Dirección:</label>");
out.println("<input type='text' id='direccion' name='direccion'>");
out.println("<label for='contrasena_actual'>Contraseña actual:</label>");
out.println("<input type='password' id='contrasena_actual' name='contrasena_actual' required>");
out.println("<label for='contrasena'>Nueva contraseña (dejar vacío para mantener la actual):</label>");

```

```

        out.println("<button type='submit' name='action' value='actualizar'>Actualizar</button>");
        out.println("<button type='submit' name='action' value='eliminar'>Eliminar Cuenta</button>");
        out.println("</form>");
        out.println("</div>");
        out.println("</body></html>");
    }

```

```

// Procesa la solicitud POST para actualizar o eliminar cuenta

```

```

@Override

```

```

protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

```

```

    // Obtener parámetros del formulario

```

```

    String action = request.getParameter("action");
    String email = request.getParameter("email");
    String nuevoNombre = request.getParameter("nombre");
    String nuevaDireccion = request.getParameter("direccion");
    String nuevaContrasena = request.getParameter("contrasena");
    String contrasenaActual = request.getParameter("contrasena_actual");

```

```

    try (Connection conn = ConexionBD.obtenerConexion()) {
        // Verificar si el usuario existe y si la contraseña es correcta
        PreparedStatement verifStmt = conn.prepareStatement(
            "SELECT id, contrasena FROM usuarios1 WHERE email = ?");
        verifStmt.setString(1, email);
        ResultSet rs = verifStmt.executeQuery();

```

```

        if (!rs.next()) {
            response.sendRedirect("UsuarioCRUD?mensaje=usuario_no_encontrado");
            return;
        }

```

```

        String contrasenaEnBD = rs.getString("contrasena");
        if (!contrasenaEnBD.equals(contrasenaActual)) {
            response.sendRedirect("UsuarioCRUD?mensaje=contrasena_incorrecta");
            return;
        }

```

```

        int usuarioId = rs.getInt("id");

```

```

        if ("actualizar".equals(action)) {
            // Actualizar datos del usuario
            String sql = "UPDATE usuarios1 SET nombre = ?, direccion = ?";
            if (nuevaContrasena != null && !nuevaContrasena.isEmpty()) {
                sql += ", contrasena = ?";
            }
            sql += " WHERE id = ?";

```

```

            PreparedStatement stmt = conn.prepareStatement(sql);
            stmt.setString(1, nuevoNombre);
            stmt.setString(2, nuevaDireccion);

```

```

            int paramIndex = 3;
            if (nuevaContrasena != null && !nuevaContrasena.isEmpty()) {
                stmt.setString(paramIndex++, nuevaContrasena);
            }
            stmt.setInt(paramIndex, usuarioId);

```

```

            int filas = stmt.executeUpdate();

```

```

            if (filas > 0) {
                response.sendRedirect("catalogo.html?mensaje=cuenta_actualizada");
            } else {

```

```

        response.sendRedirect("UsuarioCRUD?mensaje=error_actualizacion");
    }

    } else if ("eliminar".equals(action)) {
        // Eliminar cuenta del usuario
        PreparedStatement stmt = conn.prepareStatement(
            "DELETE FROM usuarios1 WHERE id = ?");
        stmt.setInt(1, usuarioId);
        int filas = stmt.executeUpdate();

        if (filas > 0) {
            // Invalidar sesión si existe
            HttpSession session = request.getSession(false);
            if (session != null) {
                session.invalidate();
            }
            response.sendRedirect("index.html?mensaje=cuenta_eliminada");
        } else {
            response.sendRedirect("UsuarioCRUD?mensaje=error_eliminacion");
        }
    }

} catch (SQLException e) {
    e.printStackTrace(); // Imprime error en consola
    response.sendRedirect("UsuarioCRUD?mensaje=error_servidor");
}
}
}

```

Fuente: Autoría propia

Cuenta HTML

En la Figura No.9 es un archivo HTML presenta una página web titulada "Gestión de Cuenta", la cual contiene dos formularios que permiten al usuario actualizar o eliminar su cuenta. El primero es un formulario de actualización de cuenta que solicita al usuario su correo electrónico (obligatorio), un nuevo nombre, una nueva dirección y una nueva contraseña (opcional). Además, incluye un campo oculto que indica que la acción que se desea realizar es "actualizar". Al enviar este formulario, los datos se envían mediante el método POST al servlet UsuarioCRUD, que procesará la solicitud.

El segundo formulario es para eliminar una cuenta. Solo solicita el correo electrónico del usuario (también obligatorio) y, de nuevo, incluye un campo oculto que indica que la acción a realizar es "eliminar". Este formulario también se envía al servlet UsuarioCRUD usando el método POST.

Ambos formularios están pensados para interactuar con un backend en Java que maneja las operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre los datos del usuario, específicamente en una tabla de base de datos llamada usuarios1. La lógica para verificar y ejecutar las acciones correspondientes se encuentra en el servlet UsuarioCRUDServlet, el cual valida el correo y la contraseña, y actualiza o elimina los datos en la base según la acción recibida.

Figura No.9 Cuenta HTML

```
<!DOCTYPE html> <!-- Define el tipo de documento como HTML5 -->
<html lang="es"> <!-- Indica que el contenido está en español -->
<head>
  <meta charset="UTF-8"> <!-- Establece la codificación de caracteres como UTF-8 -->
  <title>Gestión de Cuenta</title> <!-- Título que se muestra en la pestaña del navegador -->
</head>
<body> <!-- Cuerpo del documento -->

  <h2>Actualizar cuenta</h2> <!-- Encabezado para la sección de actualización de cuenta -->

  <!-- Formulario para actualizar la cuenta -->
  <form action="UsuarioCRUD" method="post"> <!-- Envía los datos al servlet UsuarioCRUD usando POST -->
    <input type="hidden" name="action" value="actualizar"> <!-- Campo oculto que indica la acción a realizar -->

    Correo: <input type="email" name="email" required><br> <!-- Campo para ingresar el correo, obligatorio -->
    Nuevo nombre: <input type="text" name="nombre"><br> <!-- Campo opcional para cambiar el nombre -->
    Nueva dirección: <input type="text" name="direccion"><br> <!-- Campo opcional para cambiar la dirección -->
    Nueva contraseña: <input type="password" name="contrasena"><br> <!-- Campo opcional para cambiar la contraseña -->

    <input type="submit" value="Actualizar"> <!-- Botón que envía el formulario -->
  </form>

  <h2>Eliminar cuenta</h2> <!-- Encabezado para la sección de eliminación de cuenta -->

  <!-- Formulario para eliminar la cuenta -->
  <form action="UsuarioCRUD" method="post"> <!-- También envía al servlet UsuarioCRUD por POST -->
    <input type="hidden" name="action" value="eliminar"> <!-- Campo oculto que indica acción de eliminar -->

    Correo: <input type="email" name="email" required><br> <!-- Campo para ingresar el correo, obligatorio -->

    <input type="submit" value="Eliminar cuenta"> <!-- Botón que envía el formulario para eliminar -->
  </form>
```

Fuente: Autoría propia

Conexión Con base de datos

En la Figura No.10 Es un código en Java que define una clase llamada ConexionBD dentro del paquete Tienda, cuya función es establecer una conexión con una base de datos MySQL llamada tienda, ubicada en localhost (la misma máquina). Usa el driver de MySQL para JDBC, y permite obtener una conexión a través del método estático obtenerConexion(). Si no se encuentra el driver, lanza una excepción.

Figura No.10 Conexión Con base de datos

```
// Define que esta clase pertenece al paquete llamado 'Tienda'
package Tienda;
// Importa la clase Connection para manejar conexiones a bases de datos
import java.sql.Connection;
// Importa la clase DriverManager para gestionar las conexiones JDBC
import java.sql.DriverManager;
// Importa la clase SQLException para manejar errores de base de datos
import java.sql.SQLException;
// Define la clase ConexionBD
public class ConexionBD {
    // Constante que almacena la URL de conexión a la base de datos
    private static final String URL = "jdbc:mysql://localhost:3306/tienda";
    // Constante que almacena el nombre de usuario de la base de datos
    private static final String USUARIO = "root";
    // Constante que almacena la contraseña de la base de datos (vacía en este caso)
    private static final String CONTRASENA = "";
    // Método público y estático que devuelve una conexión a la base de datos
    public static Connection obtenerConexion() throws SQLException {
        try {
            // Carga el driver de MySQL de forma explícita
            Class.forName("com.mysql.cj.jdbc.Driver"); // Carga el driver
            // Retorna una conexión utilizando la URL, usuario y contraseña definidos
            return DriverManager.getConnection(URL, USUARIO, CONTRASENA);
            // Si no se encuentra el driver, lanza una excepción SQLException personalizada
        } catch (ClassNotFoundException e) {
            throw new SQLException("No se encontró el driver de MySQL", e);
        }
    }
}
```

Fuente: Autoría Propia

Base de datos en MySQL

En la Figura No.11 es una base de datos está diseñada para gestionar una tienda online. En primer lugar, se crea una base de datos llamada "tienda" y se selecciona para su uso. La base de datos contiene tres tablas principales: usuarios1, productos y compras.

La tabla usuarios¹ se encarga de almacenar la información de los usuarios registrados. Cada usuario tiene un id único que se genera automáticamente, además de campos como nombre, email, contraseña y dirección, los cuales son necesarios para que un usuario se registre en el sistema. El campo email es único, lo que garantiza que no haya dos usuarios con la misma dirección de correo electrónico.

La tabla productos está destinada a almacenar los productos disponibles en la tienda. Al igual que en la tabla de usuarios, cada producto tiene un id único y se incluye información como el nombre, el precio (de tipo decimal con dos decimales) y una descripción opcional que permite dar más detalles sobre cada producto.

La tabla compras tiene como objetivo registrar las compras realizadas por los usuarios. Cada compra se asocia con un usuario y un producto mediante claves foráneas (usuario_id y producto_id). Además, se incluye un campo de fecha, que por defecto toma el valor del momento de la compra. Las claves foráneas aseguran que los registros en esta tabla sean válidos, vinculando cada compra a un usuario registrado y a un producto existente.

La base de datos también incluye un conjunto de instrucciones SQL para insertar algunos productos iniciales en la tabla productos (camisa, jean y zapatos) con sus respectivos precios. Al final, se incluyen comandos que desactivan temporalmente las restricciones de clave foránea para permitir eliminar todos los registros de la tabla productos y luego vuelven a activar dichas restricciones.

Figura No.11 Base de datos tienda online

```

1 • CREATE DATABASE IF NOT EXISTS tienda;
2   USE tienda;
3
4
5 • CREATE TABLE IF NOT EXISTS usuarios1 (
6     id INT AUTO_INCREMENT PRIMARY KEY,
7     nombre VARCHAR(100) NOT NULL,
8     email VARCHAR(100) NOT NULL UNIQUE,
9     contrasena VARCHAR(100) NOT NULL,
10    direccion VARCHAR(255) NOT NULL
11  );
12
13 • CREATE TABLE IF NOT EXISTS productos (
14     id INT AUTO_INCREMENT PRIMARY KEY,
15     nombre VARCHAR(100) NOT NULL,
16     precio DECIMAL(10, 2) NOT NULL,
17     descripcion TEXT
18  );
19 • SELECT * FROM usuarios1;
20
21 • CREATE TABLE IF NOT EXISTS compras (
22     id INT AUTO_INCREMENT PRIMARY KEY,
23     usuario_id INT NOT NULL,
24     producto_id INT NOT NULL,
25     fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
26     FOREIGN KEY (usuario_id) REFERENCES usuarios1(id),
27     FOREIGN KEY (producto_id) REFERENCES productos(id)
28  );
29 • SELECT * FROM compras;
30
31 -- Insertar productos
32 • INSERT INTO productos (nombre, precio) VALUES
33   ('Camisa', 50000),
34   ('Jean', 70000),
35   ('Zapatos', 100000);

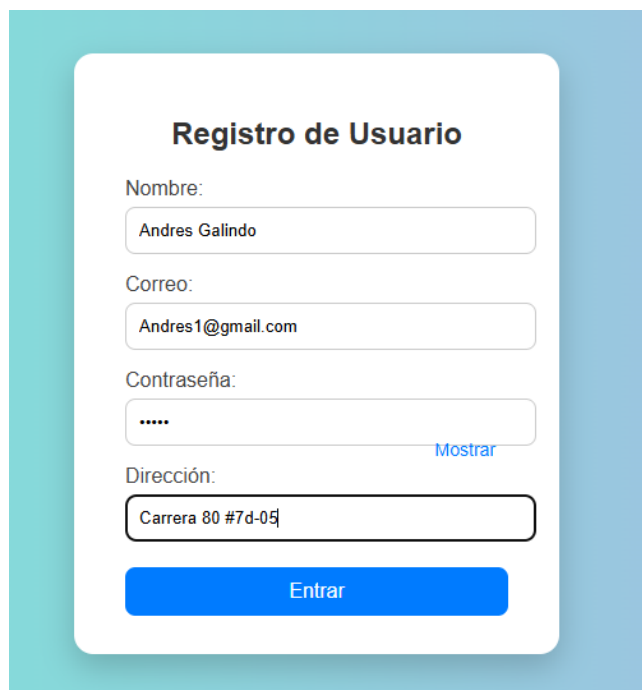
```

Fuente: Autoría Propia

Ejecución

Cuando se ejecuta el código principalmente podemos ver un login en donde el usuario se registra con datos como el nombre, correo, contraseña y dirección como se ve en la Figura No 12

Figura No.12 Registrar usuario



Registro de Usuario

Nombre:
Andres Galindo

Correo:
Andres1@gmail.com

Contraseña:
..... [Mostrar](#)

Dirección:
Carrera 80 #7d-05

[Entrar](#)

Fuente: Autoría Propia

Como podemos ver se agrega los datos a la base de datos de tienda en donde es agregado a la tabla usuario1 como se muestra en la Figura No.13

Figura No.13 Agregar a base de datos

```
18 );
19 SELECT * FROM usuarios1;
20
```

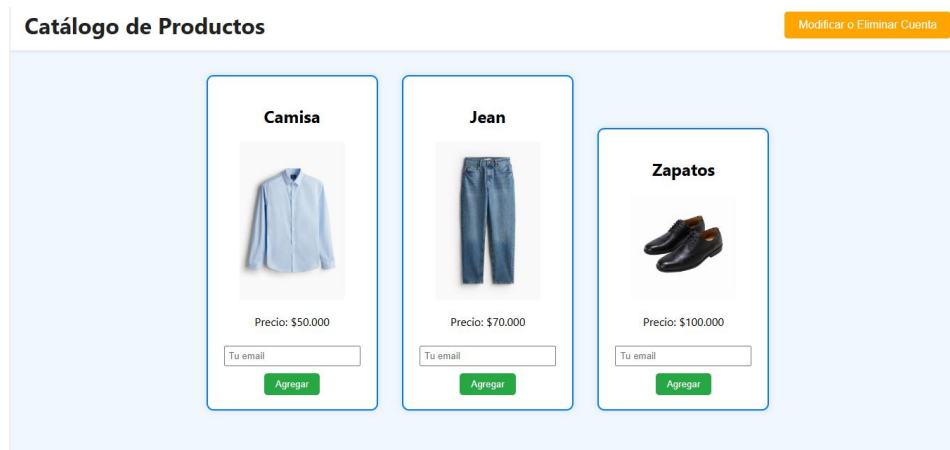
Result Grid | Filter Rows: | Edit: | Export/Import:

	id	nombre	email	contrasena	direccion
▶	1	Andres Galindo	Andres1@gmail.com	12345	Carrera 80 #7d-05
*	NULL	NULL	NULL	NULL	NULL

Fuente: Autoría Propia

Una vez ingresamos con los datos podemos ver que nos pasa a una pagina en donde muestra el catálogo de los productos de la tienda como camisa, jean y zapato como podemos ver en la Figura No.14

Figura No.14 Catálogo de Productos



Fuente: Autoría Propia

Si el usuario llega a presionar el botón de modificar o eliminar cuenta nos lleva a la siguiente pagina como se evidencia en la Figura No.15 en donde el usuario puede modificar ya sea el Email, Nombre, Dirección y la contraseña

Figura No.15 Modificar o eliminar cuenta

 The image shows a web form titled 'Modificar o Eliminar Cuenta'. It contains several input fields: 'Email' (with 'Andres1@gmail.com' entered), 'Nombre' (with 'Andres Franco' entered), 'Dirección' (with 'Carrera 80 #7d-05' entered), 'Contraseña actual' (with '*****' entered), and 'Nueva contraseña (dejar vacío para mantener la actual)' (with '*****' entered). At the bottom, there are two buttons: a blue 'Actualizar' (Update) button and a red 'Eliminar Cuenta' (Delete Account) button.

Fuente: Autoría Propia

Una vez el usuario cambia por ejemplo el nombre podemos evidenciar en la figura No.16 que el nombre en la base de datos ha sido cambiado y nos dirige nuevamente al catalogo

Figura No.16 Cambio en la base de datos

```

18  );
19  •  SELECT * FROM usuarios1;
20

```

	id	nombre	email	contrasena	direccion
▶	1	Andres Franco	Andres1@gmail.com	andres	Carrera 80 #7d-05
*	NULL	NULL	NULL	NULL	NULL

Fuente: Autoría Propia

En dado caso que el usuario quiera eliminar la cuenta debe colocar el correo electrónico y contraseña y el usuario será eliminado de la base de datos, y es dirigido al login como se evidencia en la Figura No.17

Figura No.17 Eliminar cuenta

Modificar o Eliminar Cuenta

Email:

Nombre:

Dirección:

Contraseña actual:

Nueva contraseña (dejar vacío para mantener la actual):

Fuente: Autoría Propia

Cuando el usuario quiere realizar la compra de una camisa este pone su correo electrónico y posteriormente presiona en el botón de agregar como se ve en la figura No.18

Figura No.18 Agregar compra

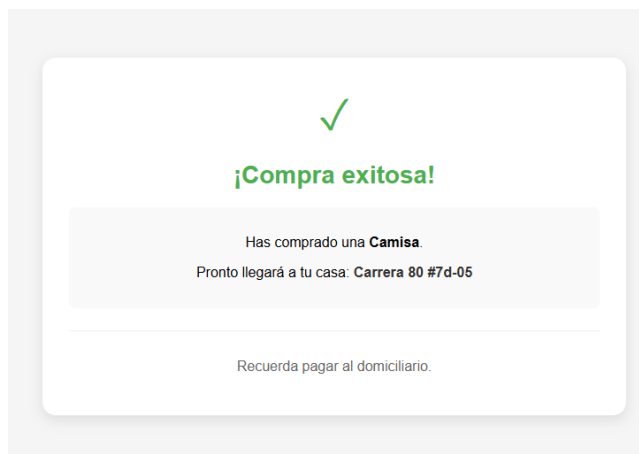


The image shows a mobile application interface for adding a purchase. It features a light blue background. In the center, there is a white rounded rectangle with a blue border. Inside this rectangle, the word "Camisa" is displayed in bold black text at the top. Below it is a photograph of a light blue button-down shirt. Under the photo, the text "Precio: \$50.000" is shown. Below the price is a text input field containing the email address "Andres1@gmail.com". At the bottom of the white rectangle is a green button with the word "Agregar" in white text.

Fuente: Autoría Propia

Finalmente, el usuario realice la compra nos muestra la siguiente pagina de compra exitosa como se muestra en la figura No.19 en donde este dato es agregado a la base de datos de la tabla compra

Figura No.19 Compra exitosa

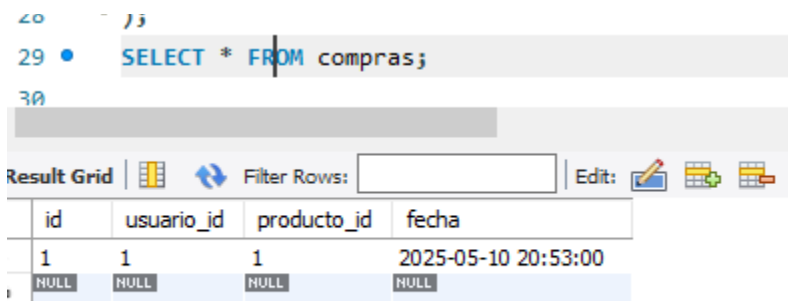


The image shows a mobile application interface for a successful purchase confirmation. It features a light gray background. In the center, there is a white rounded rectangle with a gray border. Inside this rectangle, a green checkmark is displayed at the top. Below the checkmark, the text "¡Compra exitosa!" is shown in bold green text. Below this text is a gray rounded rectangle containing the text "Has comprado una Camisa." and "Pronto llegará a tu casa: Carrera 80 #7d-05". At the bottom of the white rectangle, the text "Recuerda pagar al domiciliario." is displayed.

Fuente: Autoría Propia

Podemos visualizar en la Figura No.20 que se agrega a la tabla de compras, la compra realizada por el usuario con su respectivo id y el id del producto, también se encuentra la fecha cuando realizo la compra

Figura No. 20 Base de datos en compras



The screenshot shows a database query interface. At the top, a SQL query is entered: `SELECT * FROM compras;`. Below the query, a 'Result Grid' is displayed. The grid has a toolbar with icons for 'Filter Rows', 'Edit', and other functions. The table structure is as follows:

id	usuario_id	producto_id	fecha
1	1	1	2025-05-10 20:53:00
NULL	NULL	NULL	NULL

Fuente: Autoría Propia

Conclusión

La realización de este proyecto permitió afianzar conocimientos prácticos en el desarrollo de servicios web mediante el uso de Servlets, destacando su capacidad para gestionar peticiones y respuestas dentro de una arquitectura cliente-servidor. La implementación de páginas en HTML para el registro de usuarios facilitó la interacción inicial con el servicio, permitiendo una experiencia accesible y funcional desde casa.

Así mismo, se logró establecer una conexión exitosa con bases de datos creadas por nosotros mismos, gestionando de manera adecuada el almacenamiento y la manipulación de la información de los usuarios y productos a través de operaciones CRUD. Esta integración entre Servlets, HTML

y bases de datos permitió construir un Home Service completo y funcional, cumpliendo con los objetivos propuestos

Referencias

- El primer paso es generar un nuevo proyecto, S. D. e. C. C. un S. S. o. D. el I. N. B. (s/f). *Servlet Básico en NetBeans*. Unican.es. Recuperado el 11 de mayo de 2025, de https://personales.unican.es/corcuerp/java/slides/servlet_netbeans.pdf
- (S/f-b). Codigoxules.org. Recuperado el 11 de mayo de 2025, de <https://codigoxules.org/conectar-mysql-utilizando-driver-jdbc-java-mysql-jdbc/>
- *Conceptos básicos de HTML*. (s/f). MDN Web Docs. Recuperado el 11 de mayo de 2025, de https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Your_first_website/Creating_the_content
- *Cómo realizar operaciones de CRUD en una base de datos de MySQL*. (s/f). Google Cloud. Recuperado el 11 de mayo de 2025, de <https://cloud.google.com/integration-connectors/docs/perform-crud-operation-mysql-database?hl=es-419>

